CSCI201 Team 6 Final Project Documentation

Nairi Zeytounzian, Aiyi Wu Zheng, Luis Amezcua, Srishti Sunder,
Camerin Lee, Kelly Lukito, Humza Mahmood

Table of Contents

Project Proposal

Our project is a roommate matching program, TopTrait, that uses a Gale-Shapley algorithm to pair individuals based on their ranked lists of desirable traits. This automated system efficiently finds the most compatible roommates, saving users time and ensuring optimal living arrangements.

High-Level Requirements

We need an application that helps people find roommates easily and efficiently. Users should be able to create a profile that includes their preferences, desirable traits, and lifestyle habits, which will be used to connect them with others who share similar top traits. The program should include an engaging swipe-based system in which users can approve or disapprove of a proposed roommate. Users can then view matches and message them directly.

Technical Specifications

I - Authentication & User Profile System (14 hours)

- Implement a secure authentication system (e.g., Auth0 or Firebase Auth) with USC SSO integration.
- Design and create a comprehensive user profile system, capturing:
    - Basic Info (Name, Year, Major, Contact Info)
    - Housing Preferences (Budget, Location, Housing Type)
    - Lifestyle Traits (Sleep schedule, cleanliness, social habits, pets, smoking)
    - Desirable Traits in a Roommate
- Implement user role management (e.g., Basic User, Admin).
- Set up secure token handling and session management.
- Optional: Implement profile photo upload and validation.
- Optional: Implement account deletion and data anonymization functionality.

II - Database Architecture (18 hours)

- Design and implement a PostgreSQL/MySQL database schema with the following core tables:
    - Users (user_id, email, name, profile_data_json, created_at)
    - Traits (trait_id, trait_name, category)
    - UserTraits (user_id, trait_id, value) // Links users to their selected traits
    - Swipes (swipe_id, swiper_user_id, swiped_user_id, is_approved, timestamp)
    - Matches (match_id, user1_id, user2_id, matched_at)
    - Messages (message_id, match_id, sender_id, content, timestamp)
- Implement database security measures:
    - Prepared statements to prevent SQL injection.
    - Input validation and sanitization.
    - Encrypted connections.

III - Matching & Swipe Logic System (20 hours)

- Develop an algorithm for generating potential roommate recommendations based on:
    - Weighted Trait Matching: Compare user-submitted traits and preferences with a configurable weight for importance.
    - Preference Filters: Apply hard filters (e.g., non-smoker required, must have the same budget).
- Implement a Tinder-style swipe interface backend functionality:
    - Serve one potential match at a time from the recommendation queue.
    - Record users approve or disapprove (swipe_right/swipe_left).
- Create the match logic: A match is created only when two users have mutually approved each other.
- Optional: Implement a "Super Swipe" or priority-like feature.
- Optional: Create an admin dashboard to view matching analytics.

IV - Real-Time Chat System (16 hours)

- Implement a real-time messaging system for matched users using WebSockets
- Design and create the Messages table to store chat history persistently.
- Develop the front-end chat interface within the application.
- Implement features for a chat list and individual chat windows.
- Optional: Implement message status indicators (e.g., delivered, read).
- Optional: Support for image/file sharing within chats.

V – Web Interface (24 hours)

- Create a responsive front-end interface using HTML/CSS/JavaScript (framework TBD)
- Implement Google SSO login flow restricted to verified .edu email domains
- Page Implementations:
    - Profile Page – form-based interface to capture/edit user information and roommate preferences, persisted via API
    - Swiping Page – card-based roommate browsing interface with approve/decline actions and filtering options
    - Matches Page – list of mutual approvals retrieved from backend, with option to start conversation with real-time chat interface using WebSocket connection and message history from database
    - Settings Page – interface for updating account preferences, notification settings, and logout
- Implement error handling, loading indicators, and empty states across all pages
- Implement responsive design for mobile compatibility
- Optional:
    - Profile photo upload with client-side validation
    - Basic accessibility features (keyboard navigation, ARIA roles)
    - Client-side caching of recently fetched candidates/messages

VI - Deployment, Version Control & Testing (16 hours)

- Set up a GitHub repository with a proper branching strategy and documentation)
- Vercel full stack deployment
- Database hosting via [figure out]
- One team member will hold pseudo Tech Lead role, merging and reviewing all PR's and managing infra
- Design and implement a testing strategy:
    - Tests: For backend matching algorithms and utility functions.
- Optional: Implement a Continuous Integration/Continuous Deployment (CI/CD) pipeline using GitHub Actions

Total Estimated Core Hours: 108 hours

Detailed Design

Overview

This document describes the detailed design for TopTrait, a web application that helps university students efficiently find compatible roommates based on their lifestyle preferences and traits. The application allows users to create detailed profiles capturing basic information, lifestyle habits, and desirable roommate traits. Using a swipe-based interface, users can approve or decline potential matches, and a match is formed when two users mutually approve each other. Once matched, users can communicate through a real-time chat system. The backend will be implemented using Java with a MySQL database, and the front end will be built using React with a responsive and interactive design.

Hardware and Software Requirements

Hardware Environment:

- Backend development and deployment occur on a standard cloud or local server environment.
- Recommended minimum configuration: 2 vCPUs, 2 GB RAM, and 10 GB storage.
- Network requirements: Stable connection for real-time socket communication and database transactions.

Software Environment:

- Backend implemented in Java 17, compiled with a standard build tool (e.g., Gradle or Maven).
- Database uses MySQL 8.0 with ACID-compliant transaction support and connection pooling.
- Front-end executed in modern web browsers supporting HTML5, CSS3, and ES6.
- Real-time communication implemented via WebSocket protocol.

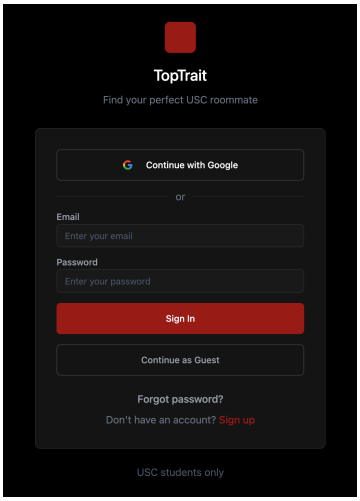
Authentication & User Profile System

Auth0 Workflow:

1. On the homepage, the user clicks "Login."

2. User can either:
    a. Sign in using personal username and password.
    b. Be redirected to login using Google (USC SSO) for authentication.
3. If login is successful, the user is redirected back to the application.
    a. If unsuccessful, invalid error messages displayed.

User vs Guest**:** Only authenticated users can save profiles, view matches, and message.



User Profile System:

On first login, the user will be presented with a profile creation page in which they must enter:

1. Basic info (name, year, major)
2. Housing & budget preferences (budget range, locations, housing type).
3. Personal lifestyle traits (personality, sleep schedule, cleanliness rating, social habits, pets, smoking).
4. Desired roommate traits with importance slider (1..5).
5. Photo upload.

Database Architecture
Database (MySQL)

- Tables
    - **Users:**
        - user_id (INT AUTO_INCREMENT) Primary key - unique ID for each user
        - email (VARCHAR(100)) - login
        - name (VARCHAR(100))
        - profile_data_json (data collected upon initial login)
            - Basic info, housing preferences, lifestyle traits, desirable traits in a roommate
        - created_at (timestamp)

- **Traits** (all available traits types)
  - traid_id (INT AUTO_INCREMENT) Primary key - unique ID per trait
  - trait_name (VARCHAR(100))
  - category (VARCHAR(100))
- **UserTraits** (links users to their selected traits)
  - user_id (INT) Foreign Key -> Users(user_id) - which user this applies to
  - trait_id (INT) Foreign Key -> Traits(trait_id) - which trait this value describes
  - value (VARCHAR(100)) (importance slider??)
- **Swipes**
  - swipe_id (INT AUTO_INCREMENT) Primary Key - unique swipe record
  - swipe_user_id (INT) Foreign Key -> Users(user_id) - user doing the swipe
  - swiped_user_id (INT) Foreign Key -> Users(user_id) - user being swiped on
  - is_approved (BOOLEAN) - TRUE if “right swipe”
  - timestamp
- **Matches**
  - match_id (INT AUTO_INCREMENT) Primary Key - unique match record
  - user1_id (INT) Foreign Key -> Users(user_id) - first user in match
  - user2_id (INT) Foreign Key -> Users(user_id) - second user in match
  - matched_at
- **Messages**
  - message_id (INT AUTO_INCREMENT) Primary Key - unique message ID
  - match_id (INT) Foreign Key -> Matches(match_id) - which match this chat belongs to
  - sender_id (INT) Foreign Key -> Users(user_id) - who sent message
  - content - actual message
  - timestamp

Key Classes:

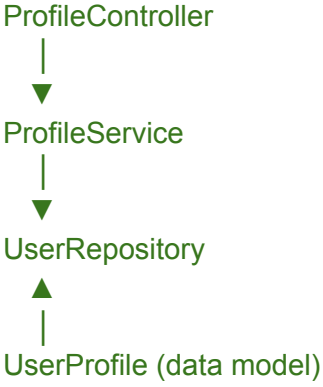
Class	Purpose	Key Methods
`UserProfile`	Data model storing all user-entered information (name, year, major, housing preferences, traits).	`toJSON()`, `validate()`, `updateTraits()`

`ProfileController`	Receives client HTTP requests and handles serialization/deserialization of JSON payloads.	`handleProfileSubmit(Request req)`, `getProfile(int userId)`
`ProfileService`	Applies business logic for creating, updating, or deleting profile data. Ensures all dependent tables remain consistent.	`saveProfile(UserProfile p)`, `updateProfile(UserProfile p)`, `getProfileById(int id)`
`UserRepository`	Provides low-level database access methods and SQL queries using prepared statements.	`insertUser()`, `updateUser()`, `fetchUser()`

Class Diagram

The following diagram illustrates the main object-oriented relationships within the Profile and Messaging modules:

Controller → Service → Repository structure:



Relationships:
- ProfileController invokes ProfileService for validation and business logic.
- ProfileService calls UserRepository to perform database operations.
- UserRepository returns UserProfile objects that encapsulate user data.

For the Messaging module:
MessageController → MessageService → MessageRepository handles asynchronous threaded message delivery.

Data Collection System
The Data Collection System handles how user data and preferences are captured, stored, and retrieved throughout the application. When a user first logs in, they are prompted to create a profile containing their basic information.

Workflow:

- On initial login, a new user record is created in the database.
    - Upon successful authentication, the backend checks whether the user already exists in the Users table.
    - If no record exists, the system instantiates a new UserProfile object and inserts a placeholder entry into the database with the user's unique ID and email.

- The user completes a profile creation form, providing all required information.

- Input validation occurs on two levels to ensure correctness
    - Client-side validation prevents submission of incomplete or malformed data using pattern and range constraints.
    - Server-side validation uses strict type checks and validation annotations in the backend (@NotNull, @Range, @Pattern) to enforce consistency.

- When the user submits the form, the information is serialized and stored in the user's profile record. The system serializes all fields into a JSON payload. This payload is transmitted to the /api/profile/update endpoint using an asynchronous POST request.

- If the user edits their profile later, existing values are retrieved, displayed, and updated upon re-submission.
    - If a returning user edits their profile, a GET request retrieves the stored data using the user's unique ID. The backend serializes the existing record to JSON and returns it to populate the form.
    - Upon re-submission, the updated data triggers a merge operation, which overwrites only modified fields while preserving existing records.

Data Relationships:

- The system maintains a master list of all possible lifestyle and personality traits.
    - Each user is linked to one or more traits via the UserTraits mapping table.

- The UserTraits table includes a numeric importance weight (1–5) for each trait, representing how strongly the user values that characteristic in themselves or in a roommate.
- During the matching process, the system performs join operations on UserTraits to compare overlapping traits between two users.

Matching & Optimization System

The roommate matching engine applies a modified Gale–Shapley stable matching algorithm to optimize compatibility scores between users.

Each user's ranked roommate preferences are derived from weighted trait similarities, ensuring stable matches where no pair of users would prefer each other over their current match.

```
Initialize all users as unmatched
while exists user U who is unmatched and has not proposed to every candidate:
    V = highest-ranked candidate for U who U has not yet proposed to
    if V is unmatched:
        pair(U, V)
    else if V prefers U over current match:
        unmatch(V's current partner)
        pair(U, V)
    else:
        mark proposal(U, V) as rejected
return all matched pairs
```

Notes:

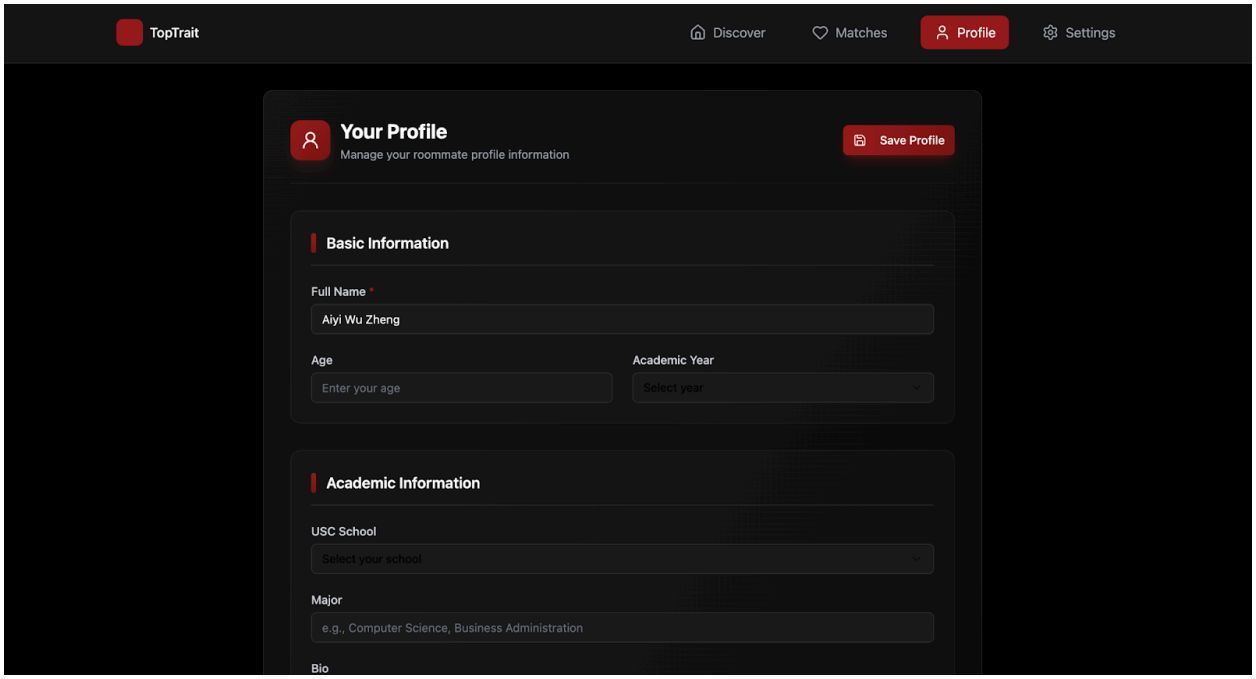
- Each user maintains a ranked list of candidates computed from the weighted similarity of traits.
- "Prefers" is determined by higher compatibility scores between users.
- A match is finalized only when both users mutually approve each other through the swiping interface.

Web Interface

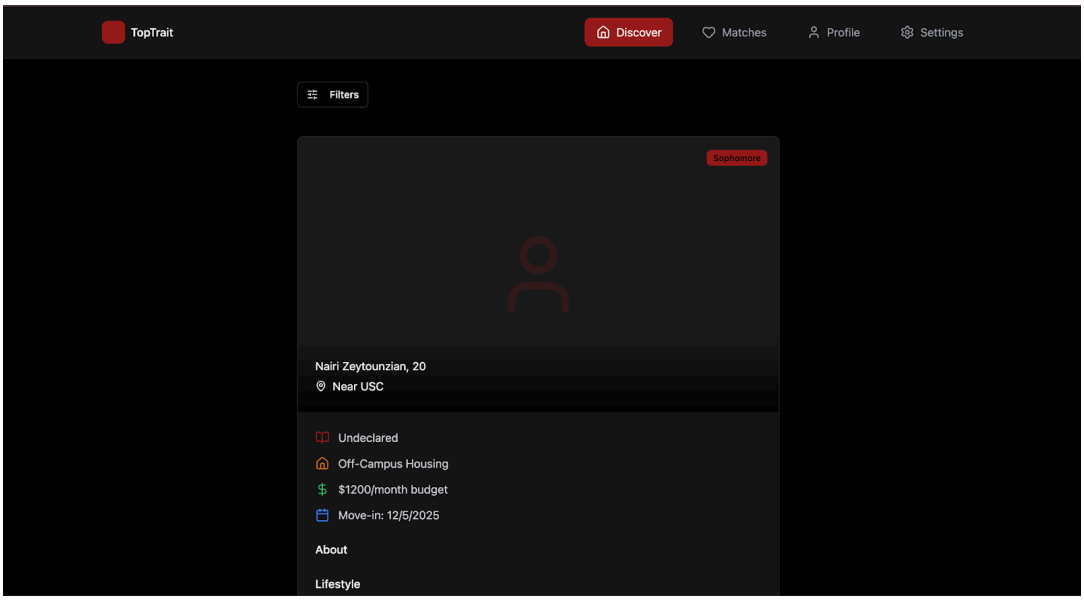
The Web Interface provides all user-facing pages and supports interactive behavior for viewing, inputting, and responding to information within the system.

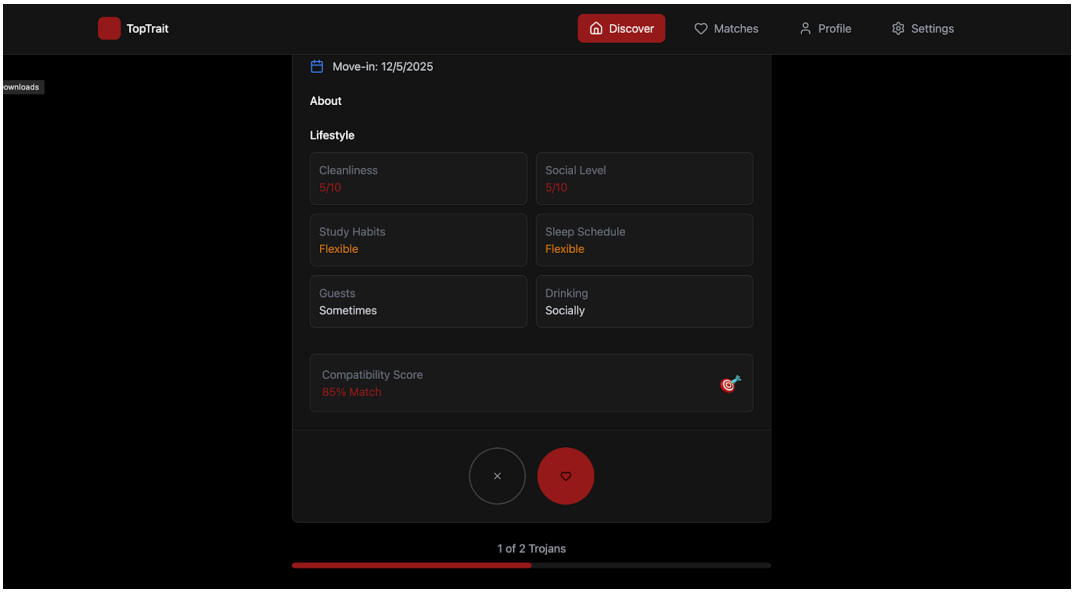
Key Screens:

- **Profile Page**
  - Displays fields for basic information, housing preferences, and lifestyle traits.
  - Composed of interactive form elements bound to internal state variables that represent user profile attributes.
  - On form submission, a POST request transmits serialized profile data to the backend, which updates the user record in the database.
  - The page dynamically retrieves stored profile data on load through a GET endpoint, ensuring users can edit previously saved information

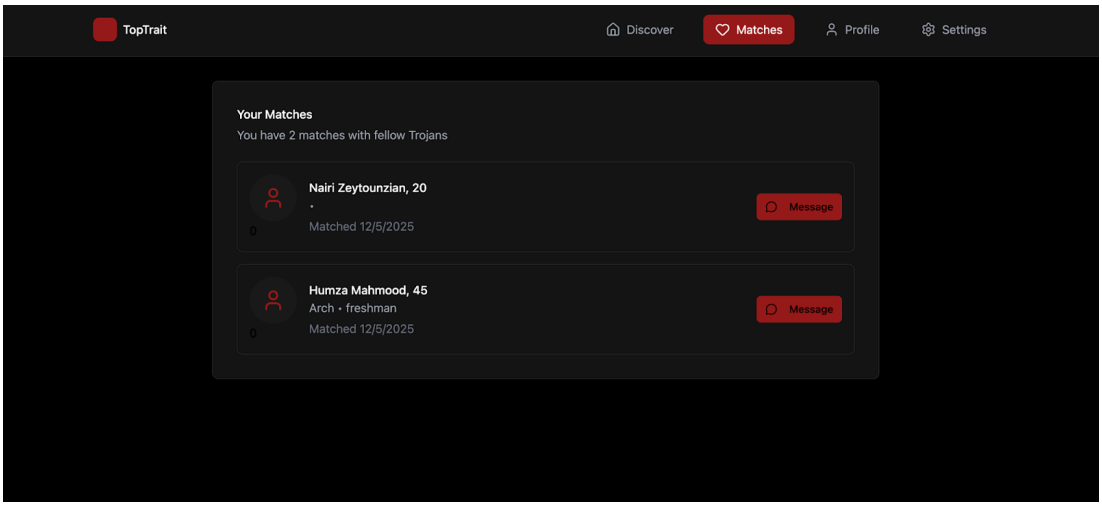


- **Swiping Page**
  - Shows one potential roommate candidate at a time.
  - Each profile is generated by retrieving a candidate object from the matching queue, which includes the candidate's traits, preferences, and summary information.
    - Provides two large action buttons for "Approve" and "Decline" that trigger event handlers that record the swipe action through an API call to the backend.
  - Includes loading indicators between swipes and feedback animations for user actions.





- **Matches Page & Messaging Page**
    - Lists all users with whom mutual approval has occurred.
    - Each match entry includes a preview of the user's name and key traits.
    - Selecting a match triggers a navigation event to the messaging view while maintaining the current application state (e.g., logged-in user and session token)
    - Populates a list view from the current user's match dataset obtained via a GET request to the matches endpoint
    - New messages appear instantly and are timestamped.
    - Unread messages are visually distinguished.
    - On the server side, a dedicated thread pool handles concurrent message exchanges across all active sessions
    - On the client interface, an event listener thread continuously listens for new message events from the server.

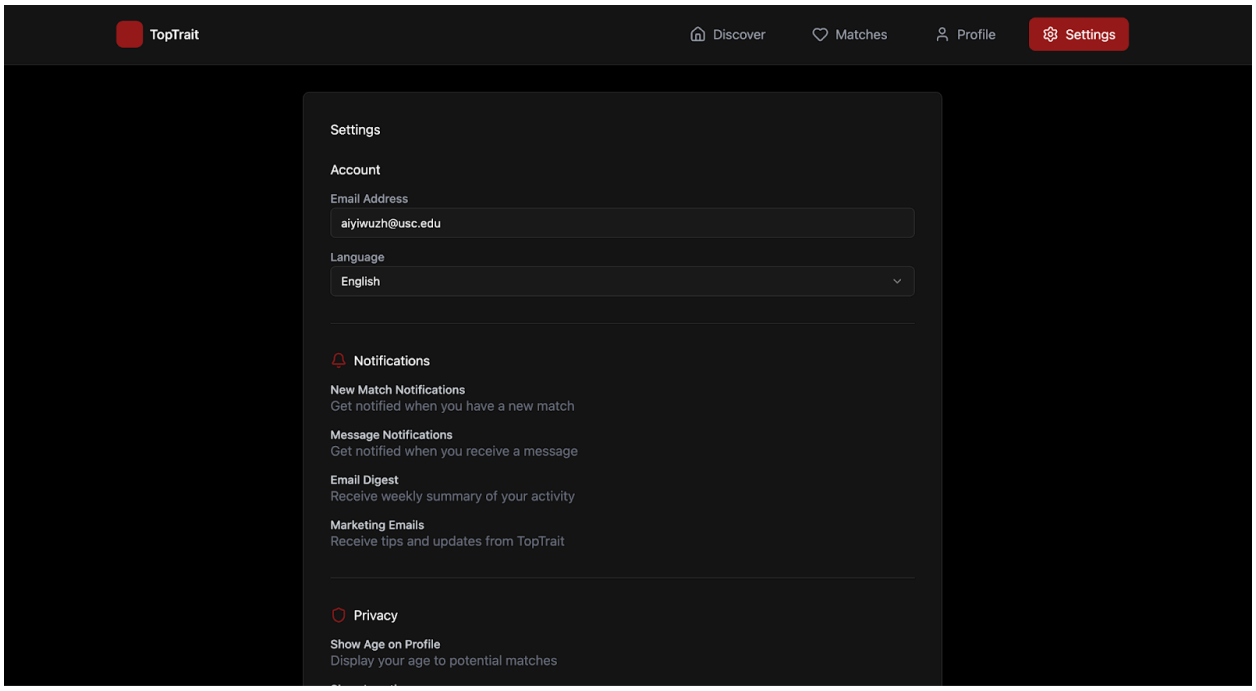



```
### Pseudocode – Messaging Thread Handling

Thread listener = new Thread(() → {
    while (socket.isOpen()):
        message = socket.receive()
        updateChatUI(message)
})

Thread sender = new Thread(() → {
    while (!outgoingQueue.isEmpty()):
        msg = outgoingQueue.poll()
        socket.send(msg)
        saveToDatabase(msg)
})
```

- **Settings Page**
    - Provides controls for account management functions, including password update, notification toggles, and account deletion.
    - Includes confirmation dialogs for sensitive actions.



The following diagram shows the navigation flow between major pages:

[Profile Page] → [Swiping Page] → [Matches Page] → [Messaging Page] → [Settings Page]

Testing Plan

I - Authentication User Profile System

<u>Test Case 1 - Black Box Testing - Login - Successful Authentication and Login</u>

Input:

- After entering a valid email and password, the user clicks “Sign In.”

Expected Output:

- The user is redirected to the homepage of the application system.
- The user’s profile data is fetched from the backend and accessible from the profile page.

Test Case 2 - Black Box Testing - Login - Invalid Email

Input:

- The user attempts to log in with an invalid email address.

Expected Output:

- The user fails to log in and stays on the login page. An error message that says “Invalid email address” is displayed.

Test Case 3 - Black Box Testing - Login - Non-USC Email Address

Input:

- The user attempts to log in with an email that does not end in “@usc.edu.”

Expected Output:

- The user fails to log in and stays on the login page. An error message that says “Email must be a valid USC email address” is displayed.

Test Case 4 - Black Box Testing - Login - Invalid Password

Input:

- The user enters an invalid password and clicks “Sign In.”

Expected Output:

- The user fails to log in and stays on the login page. An error message that says “Invalid password” is displayed.

Test Case 5 - Black Box Testing - Login - Successful USC SSO Login

Input:

- The user clicks “Continue with Google” for login and they’re redirected to another page to select their USC Google Account. The user logs into their USC Google Account with the correct email and password.

Expected Output:

- Once logged into their USC Google account, they are redirected to the homepage of the application and login is successful.

Test Case 6 - Black Box Testing - Login - Invalid USC SSO Login

Input:

- The user clicks “Continue with Google” for login and they’re redirected to another page to select their USC Google Account. The user attempts to log into USC Google Account but enters incorrect login information for the account.

Expected Output:

- The user fails to login through USC SSO due to failed USC Google Account login. They are redirected back to the login page and can try again either through web page log in or USC SSO. An error message that says “USC SSO failed. Please try again” is displayed.

Test Case 7 - Black Box Testing - Creating an Account - Invalid Email

Input:

- The user clicks “Sign Up” and enters an invalid email address.

Expected Output:

- The user fails to create an account and stays on the signup page. An error message that says “Invalid Email Address” is displayed.

Test Case 8 - Black Box Testing - Creating an Account - Non-USC Email

Input:

- The user clicks “Sign Up” and enters a non-USC email address.

Expected Output:

- The user fails to create an account and stays on the signup page. An error message that says “Email address must be a valid USC email” is displayed.

Test Case 9 - Black Box Testing - Creating an Account - Invalid/Weak Password

Input:

- The user clicks “Sign Up” and enters a password that does not meet requirements (therefore is invalid and weak). A password is considered weak if it fails to meet any of the requirements:
    - 8 or more characters long
    - At least 1 uppercase letter
    - At least 1 lowercase letter
    - At least 1 number (0-9)
    - At least 1 special character (!,@,#,$,%,^,&,*,(,))

Expected Output:

- The user fails to create an account and stays on the signup page. An error message that says “Invalid password” is displayed.

Test Case 10 - Black Box Testing - Creating an Account - Duplicate Account Creation

Input:

- The user clicks “Sign Up” and enters an email address that is already registered.

Expected Output:

- The user fails to create an account and stays on the signup page. An error message that says “Email already taken” is displayed.

Test Case 11 - Stress Testing - High Login Frequency

Input:

- Simulate 1000 users attempting to log in simultaneously with both the website login page and USC SSO. Users should be trying to sign up for a new account, logging in with the website log-in, or using the USC SSO.

Expected Output:

- The authentication system should handle all login requests smoothly without crashing.
- All users should be authenticated successfully within 5 seconds.

Test Case 12 - Black Box Testing - Login - Locked Account

Input:

- The user attempts to log in with an incorrect username and password multiple times which causes the system to lock their account.

Expected Output:

- The user fails to log into their account and stays on the login page. An error message that says “Account locked due to too many failed login attempts. Please try again in 30 minutes” is displayed.

Test Case 13 - Black Box Testing - Password Reset - Successful Password Reset

Input:

- The user clicks “Forgot Password?” and is redirected to a page where they can enter their email address to request a password reset. The user follows instructions in their email to successfully reset their password.

Expected Output:

- The user successfully resets their password and receives a confirmation email that their password has been reset. They are now able to successfully log in with the new password.

Test Case 14 - Security Testing - Session Management

Input:

- User logs in successfully, then closes the browser without logging out. Later, they reopen the browser and navigate to the application URL.

Expected Output:

- The user should either be still logged in (if using persistent sessions) or be redirected to the login page (if sessions expire). Sensitive profile data should not be exposed.

II - User Profile System

Test Case 1 - Black Box Testing - Successful Profile Creation

Input:

- A newly registered user fills out all the required profile fields: name, year, major, housing preferences, lifestyle traits, and desirable roommate traits (top trait)
- Then user clicks “Save”

Expected Output:

- The profile information is saved successfully
- A confirmation message like “Profile updated successfully” appears (both on initial profile creation and modifications/changes made)
- All data appears correctly on the profile page upon reload

Test Case 2 - Black Box Testing - Missing Required Field

Input:

- A user leaves a required field blank and clicks “Save”

Expected Output:

- The form does not submit
- A red warning appears under the missing field “This field is required”
- No changes are saved to the database

Test Case 3 - Black Box Testing - Invalid Budget Input

Input:

- The user enters non-number text in the Budget field and clicks “Save”

Expected Output:

- An error message appears “Please enter a valid number”
- The system prevents saving invalid data

Test Case 4 - Black Box Testing - Invalid Contact Info Format

Input:

- The user enters an improperly formatted email or phone number in the contact information field

Expected Output:

- The system displays an inline validation message “Please enter a valid email address”
- Data is not saved until corrected

Test Case 5 - Black Box Testing - Trait Selection and Save

Input:

- The user selects multiple lifestyle traits and clicks “Save”

Expected Output:

- All selected traits and their values are saved to the UserTraits table
- Upon revisiting the profile page, previously selected traits are pre-filled correctly

Test Case 6 - White Box Testing - Database Integration

Input:

- After saving the profile, query the Users and UserTraits tables

Expected Output:

- Entries exist for the user in both tables with correct data relationships
- JSON data in Users.profile_data_json matches the submitted profile values

Test Case 7 - Regression Testing - Edit and Re-Save Profile

Input:

- A user updates their previously saved profile

Expected Output:

- The database updates the corresponding fields correctly
- Reloading the profile page shows the new data
- Old data is overwritten, not duplicated

Test Case 8 - Security Testing - Unauthorized Profile Access NEED????

Input:

- A non-authenticated (guest) user attempts to access profile via direct URL

Expected Output:

- The system redirects them to the login page
- No profile data is exposed in network responses

Test Case 9 - Black Box Testing - Account Deletion

Input:

- A user clicks “Delete Account” and confirms

Expected Output:

- All associated data in Users, UserTraits, Swipes, and Matches referencing that user_id are deleted
- The user is redirected to the homepage with a confirmation message “Account deleted successfully”

Test Case 10 - Stress Testing - Concurrent Profile Saves

Input:

- Simulate 100 users editing and saving profiles simultaneously

Expected Output:

- No data corruption or overwriting occurs
- Each user’s data is stored under the correct user_id
- Average response time remains under 2 seconds

Test Case 11 - Integration Testing - Profile -> Matching

Input:

- After profile creation, the user proceeds to the matching/swiping interface

Expected Output:

- Matching algorithm correctly accesses and uses profile and trait data from the database
- No “missing data” or “null field” errors occur

Test Case 12 - Usability Testing - Profile Page UI

Input:

- Users navigate the profile form on desktop and mobile browsers

Expected Output:

- All fields are visible and usable
- Dropdowns and sliders function smoothly
- Buttons are responsive and display confirmation states

III - Database Architecture

Test Case 1 – White Box Testing – Database Connection Failure

Input:

- Temporarily disconnect the backend server from the database or use invalid credentials.

Expected Output:

- The system fails to connect gracefully.
- A log entry records “Database connection error.”
- The application displays an error message such as “Service unavailable. Please try again later.”
- The web server does not crash.

Test Case 2 – Black Box Testing – Data Persistence and Retrieval

Input:

- The user creates a new account and fills in profile data (name, email, lifestyle preferences).

- The user logs out, then logs back in.

Expected Output:

- The same profile data is fetched from the database correctly.
- All user fields (e.g., name, email, preferences) match the original input exactly.

Test Case 3 – Regression Testing – Database Schema Update

Input:

- After a database schema update (adding new columns to “Users” table), run previous test scripts that query user data.

Expected Output:

- All previous user data remains accessible.
- Legacy functions dependent on the “Users” table still work correctly without crashes or missing values.

Test Case 4 - White Box Testing - Data Integrity on Delete

Input:

- Delete a user who has existing swipes, matches, and chat messages.

Expected Output:

- All associated records in Swipes, Matches, and ChatMessages tables are also deleted (via CASCADE delete) or anonymized. No orphaned records remain.

Test Case 5 - Stress Testing - Concurrent Writes

Input:

- Simulate 50 users updating their profiles simultaneously.

Expected Output:

- The database handles all transactions without deadlocks. All updates are persisted correctly without data corruption.

IV - Matching & Swipe Logic System

Test Case 1 – Black Box Testing – Successful Match

Input:

- User A swipes right on User B.
- User B swipes right on User A.

Expected Output:

- Both users receive a “You’ve got a match!” message.
- The match is recorded in the database under each user’s match list.

Test Case 2 – Black Box Testing – No Match

Input:

- User A swipes right on User B.
- User B swipes left on User A.

Expected Output:

- No match is created in the system.
- Neither user receives a match notification.

Test Case 3 – White Box Testing – Matching Algorithm Verification

Input:

- Provide two user profiles with known compatibility scores (e.g., 0.95 similarity vs. 0.40 similarity).

Expected Output:

- The algorithm correctly prioritizes higher compatibility pairs.
- The function returns expected matching probabilities within tolerance levels.

Test Case 4 - Black Box Testing - Self-Swipe Prevention

Input:

- User A attempts to swipe on their own profile.

Expected Output:

- The swipe is not registered. The system displays a message: "You cannot swipe on your own profile."

Test Case 5 - Black Box Testing - Duplicate Swipe Prevention

Input:

- User A swipes right on User B multiple times.

Expected Output:

- Only the first swipe is recorded. Subsequent swipes are ignored or the user sees "You have already swiped on this user."

Test Case 6 - White Box Testing - Compatibility Score Calculation

Input:

- Create two user profiles with specific, known trait overlaps (e.g., same major, similar sleep schedule, same cleanliness level).

Expected Output:

- The matching algorithm calculates a compatibility score that accurately reflects the predefined overlaps and weights.

Test Case 7 - Black Box Testing - Match List Ordering

Input:

- Users have multiple matches with varying compatibility scores.

Expected Output:

- The match list is displayed in descending order of compatibility score (highest matches first).

Test Case 8 - Stress Testing - Swipe Spam

Input:

- Simulate a user swiping right on 1000 profiles in rapid succession.

Expected Output:

- The system processes all swipes without significant performance degradation. Match creation still occurs correctly for mutual swipes.

V - Real-Time Chat System

Test Case 1 – Black Box Testing – Message Delivery
Input:
- User A and User B are matched.
- User A sends “Hello!” in the chat window.

Expected Output:
- User B receives “Hello!” within 1 second.
- The message is saved to the database and visible in chat history for both users.

Test Case 2 – Stress Testing – High Volume Chat Traffic
Input:
- Simulate 50 active user pairs sending messages simultaneously.

Expected Output:
- The chat server handles all connections without downtime or message loss.
- Average message delivery time remains under 2 seconds.

Test Case 3 – Black Box Testing – Network Disconnect
Input:

- User A loses internet connection mid-chat.

Expected Output:
- The chat interface displays “Connection lost.”
- Upon reconnection, pending messages are sent automatically.

Test Case 4 - Black Box Testing - Chat Between Matched Users Only

Input:

- User A tries to send a message to User C, with whom they are not matched, by directly calling the chat API.

Expected Output:

- Message is not delivered. The system returns an error: "You can only message your matches."

Test Case 5 - Black Box Testing - Message History Persistence

Input:

- User A and User B exchange messages, then both log out and back in later.

Expected Output:

- The entire chat history is preserved and displayed correctly upon returning to the chat.

Test Case 6 - Usability Testing - Typing Indicators & Read Receipts

Input:

- User A is typing a message in a chat with User B.

Expected Output:

- User B sees a "typing..." indicator. When User A sends the message, User B sees it delivered/read.

VI – Web Interface

Test Case 1 – Black Box Testing – Responsive Design
Input:

- Open the website on a laptop, a smartphone, and a tablet.

Expected Output:

- All elements (buttons, text, chat windows) resize properly and remain fully visible.
- No visual distortion or text overflow occurs.

Test Case 2 – Accessibility Testing – Screen Reader Compatibility

Input:

- Navigate the site using a screen reader and keyboard-only input.

Expected Output:

- Each button, link, and input field is properly labeled.
- The user can navigate through all features using only the keyboard (Tab, Enter, Space).

Test Case 3 – Regression Testing – UI Update Verification

Input:

- After updating the home page layout, test all buttons and links from the previous version.

Expected Output:

- All existing navigation routes still work correctly.
- No broken links or missing assets appear after deployment.

Test Case 4 - Cross-Browser Compatibility

Input:

- Access the application on Chrome, Firefox, Safari, and Edge.

Expected Output:

- All functionality (swiping, chatting, profile editing) works consistently across all browsers. No console errors appear.

VII - End-to-End & Integration Tests

Test Case 1 - E2E - Complete User Journey: Signup to Match to Chat

Input:

1. New users sign up successfully.
2. Users fill out their profile completely.
3. User swipes on several profiles.
4. The user gets a match.
5. The user sends a message to the match.

Expected Output:

- Each step completes successfully, with data flowing correctly from the frontend to the backend database and between systems (Profile -> Matching -> Chat).

Test Case 2 - Integration - Profile Update Affects Matching

Input:

- A user updates their "Top Trait" in their profile after already being in the matching pool.

Expected Output:

- The matching algorithm immediately uses the new trait data to recalculate compatibility for future potential matches. Existing matches are unaffected.

Test Case 3 - Integration - Unmatch Scenario

Input:

- User A "unmatches" User B from their match list.

Expected Output:

- The match record is removed (or marked inactive).
- The chat between User A and User B becomes inaccessible.
- Both users are removed from each other's match lists.

Test Case 4 - Performance - System Load under Peak Usage

Input:

- Simulate 500 active users performing mixed actions: 30% logging in, 40% swiping, 20% chatting, 10% editing profiles.

Expected Output:

- The system remains stable. API response times stay under 3 seconds. No services crash.

Deployment Documentation

Step 1: Ensure deployment environment is ready for deployment
- Use a server with minimum 2 vCPUs, 2 GB RAM, 10 GB storage, and stable networking for WebSocket and database communication.
- Ensure Java 17, Maven/Gradle, and MySQL are installed on the server.

Step 2: Build and deploy the Java Backend
- Compile the backend using Maven/Gradle and upload the generated .jar file to the live server.
- Start the server.

Step 3: Configure environment variables
- Set database URL, authentication keys, APIs, WebSocket configuration on the live server.

Step 4: Run MySQL database and ensure that all data was migrated successfully
- Run verify.sql script

Step 5: Deploy front-end
- Push the front-end to Vercel and update the API base URL to point to the deployed Java backend

Step 6: Perform test on real-time communication features
- Confirm that WebSocket connections are functioning by sending test messages between two clients.

Step 7: Perform a full regression test of the live application
- Verify login, profile creation, swiping, matching, and messaging work correctly

Step 8: Notify users of the updated application and provide them with the URL for access.

AI Logs

Nairi Zeytounzian

1. Frontend UI & Functionality
    a. Prompt: Upon creating the frontend, “embed in the login page, a sign up button for users that are going to be signing in for the first time. Make sure the sign up page follows the current theme of the rest of the login page but asks for more information that a typical sign up page would. Also create the ability to continue as a guest.”
    Prompt: “When users press log out, it should redirect them to home sign in page”
    Prompt: “When the user clicks delete account, they should be directed to home sign in”
    Prompt: “Fix the google auth so that the user is redirected to log in page with google and usc sso.”
    Prompt: “Fix log in to where username and password works for anyone in the database.”
    Worked on configuring frontend, linkage between files
    Worked on database in supabase
    b. Issues: There were a lot of issues with the necessary components a typical login page needs to include.
        - Users weren’t being redirected after logging in, signing in, or delete account
        - Signing in wasn’t asking for a strong password, password could be extremely weak.
    c. Fix: Prompt: “Add the ability to embed “show password” or “hide password” when signing up so that the user can see the password they are typing. Passwords need to be 8 characters and include 2 special characters. Once they have entered a valid password, add a green checkmark next to the password requirements indication that they made a valid password.
    d. Explanation: The authentication experience was redesigned to flow logically between screens, support secure credential input, and provide additional onboarding for roommate-matching profile data. Guest mode and password validation features improve accessibility and onboarding while staying consistent with UI theming.

Humza Mahmood

1. Backend and Database Connection
    a. Prompt: “Implement a robust backend that supports secure authentication, scalable profile storage, match creation, and real-time messaging. Expand the current database design by adding multiple new relational tables including Matches, Messages, UserTraits, and Swipes, ensuring referential integrity via foreign key constraints and optimized query indexing. Write secure migration scripts to evolve the schema without data loss.”

b. “Introduce multithreading into the backend architecture so multiple users can perform match operations, send messages, and swipe concurrently without blocking or performance degradation. Apply thread-safe data structures and protect critical sections to avoid race conditions during mutual match creation.”
c. “Create and deploy a WebSocket-based messaging subsystem enabling persistent, real-time communication between two matched users. Messages must be reliably delivered, ordered correctly, and stored into the Messages table so users can access chat history upon reconnection.”
d. “Integrate backend logic with Supabase authentication and enforce secure access control rules on private endpoints. Provide endpoints for user profile retrieval, trait scoring, match generation, and message sending with full error handling and request validation. Ensure the system can scale under high usage during peak student matching cycles.”

2. Issues: Connecting users was failing
- Some endpoints required additional authentication state checks
- Initial messaging service required better synchronization to avoid message duplication

Aiyi Wu Zheng

1. Frontend UI & Functionality
    a. Prompt: “Create a responsive front-end interface for a USC roommate matching web application called TopTrait with the following page implementations: Profile Page: form-based interface to capture/edit user information and roommate preferences, persisted via API, swiping page: card-based roommate browsing interface with approve/decline actions and filtering options, matches page: list of mutual approvals retrieved from backend, with option to start conversation, 1-to-1 messaging page: real-time chat interface using WebSocket connection and message history from database, settings page: interface for updating account preferences, notification settings, and logout.”
    b. Issues: The application did not seem to be tailored to USC students and needed more specific questions such as school year, majors, study habits, etc.
    c. Fix: Gave the prompt: “Can you tailor the application more to USC students who are looking to find other USC roommates? Make sure to include school specific questions.”
    d. Explanation: Adding additional information that is relevant to being a USC student allows students to get to know other people better and ensure that they are rooming with someone who also attends USC. This makes the application more tailored.
2. Frontend UI & Functionality
    a. Prompt: “On each match, include a preview section of a potential match by listing the top 3 traits shared between the two people.”
    b. Issues: The application did not have the traits on the actual discovery card of every potential match. Instead, it put it on the matches page where it would only show the top traits shared after two people matched.
    c. Fix: Gave the prompt: “Add the top 3 traits shared section on the discover card of the potential match on the discover page.”

   d. Explanation: Allowing the users to get a quick description of the top three shared traits between them and the potential match will make it easier for them to identify if the person would be a good match for them.
3. Log In and Google Authentication UI
   a. Prompt: “For the login page, can you make sure to add two options? One for logging in with an email and password and another option for logging in through Google authentication?”
   b. Issues: The correct log in buttons were added but there was no error message displayed when a user failed to log in correctly.
   c. Fix: “Please add an error message for when the user fails to log in.”
   d. Explanation: Providing the correct error messages displayed on the frontend allows the user to understand what went wrong and try logging in again.
4. Cover Page UI
   a. Prompt: “Can you make a cover page that loads when a user goes onto the site? It should say "TopTrait” and should go before the log in page. Make it visually appealing and eye-catching.”
   b. Issues: There was just a button on the page with either log in or sign up but I wanted the functionality to be more user-friendly to where if they pressed anywhere on the screen, they would be redirected to the log in page.
   c. Fix: Gave prompt: “Change it to where that if the user clicked anywhere on the screen, they would be redirected to the login page.”
   d. Explanation: Being able to press anywhere on the screen is more user-friendly and interactive.

Srishti Sunder

1. Frontend Logic & Application Flow
   a. Prompt: “Fix the issue where logging out does not reliably reset the application state. Ensure that after logout, the user is always returned to the login screen, Guest Mode is cleared, and navigation resets to the correct default page. The app should also avoid stale UI states that persist after signout.”
   b. Issues: Logging out did not consistently clear key UI state variables. Guest Mode sometimes persisted after signout, and the user was occasionally redirected back to a previously open page (such as Matches or Profile) instead of the Login page. The reset logic was split between the manual logout handler and the onAuthStateChange listener, causing timing conflicts and inconsistent behavior.
   c. Fix:
      i.
```
const handleLogout = async () => {
  await supabase.auth.signOut();
  setIsAuthenticated(false);
  setIsGuest(false);
  setAuthPage('login');
  setCurrentPage('swiping');
};
```

   d. Explanation: Before, logout didn't fully "wipe the slate clean," so parts of the old session stuck around. Now everything resets in one place, so logging out always looks the same and works the way users expect.
2. Frontend Logic & Application Flow
   a. Prompt: "Fix the logout behavior so the app always returns to the correct starting screen and clears all user states properly."
   b. Issues: After logging out, the app sometimes stayed on the previous page—like Matches or Profile—and occasionally kept Guest Mode active even though the session had ended. This happened because different parts of the app were trying to reset state at the same time, causing inconsistent results.
   c. Moved all logout-related resets into one unified function so every part of the app updates together.

```
const handleLogout = async () => {
  await supabase.auth.signOut();
  setIsAuthenticated(false);
  setIsGuest(false);
  setAuthPage('login');
  setCurrentPage('swiping');
};
```

   d. Explanation: Putting everything in one place ensures that logging out always resets the app the same way. Now the user is taken back to the login screen, Guest Mode turns off, and no old pages linger.
3. Styling Architecture & Tailwind Integration
   a. Prompt: "Separate the visual styling from the component logic by moving styles into dedicated CSS files where possible, while also integrating Tailwind for utility-based styling. Ensure that the app actually uses the extracted CSS and Tailwind classes correctly."
   b. Issues: After dividing the CSS and the component TSX files, some of the separated styles were not being applied because the project relied heavily on Tailwind classes rather than traditional CSS imports. Several extracted CSS rules were overwritten by Tailwind defaults, or were never imported into the component tree. This caused parts of the UI, like the cover animations, login card, and matches layout, to appear inconsistent or styled only partially. Additionally, Tailwind's precedence made it unclear which styles "won" when both CSS and utility classes were present.
   c. Fix: Verified which components still relied on external CSS and explicitly imported the necessary files. For components that were already using Tailwind extensively, consolidated duplicated or conflicting CSS into Tailwind classes for consistency. Cleaned up unused selectors and reorganized styles so each component now uses either Tailwind or scoped CSS intentionally, instead of both colliding
   d. Explanation: By clarifying which styles live in CSS files and which are handled by Tailwind, the UI now renders consistently across components. Tailwind handles fast layout and spacing, while animations, gradients, and custom effects remain

in external CSS where necessary. This prevents style conflicts and ensures that extracted CSS actually gets used.